GENERALITAT VALENCIANA

ies SERPIS | departament informàtica

UNIDAD 7.2. CONTROL DE ERRORES EN JAVASCRIPT

Desarrollo de aplicaciones web en entorno cliente · Curso 26/27

2º DAW

Rafael Torres Ferrer

¿Qué pasa aquí?

```
console.log("Inicio");
console.log(e);
console.log("Fin");
```

La variable **e** no existe. ¿Se imprime “Fin”?

Pista: ¿qué hace el intérprete cuando encuentra un error?

Hoy aprenderemos a

1. **Clasificar**
   los tipos de errores en un programa

2. **Lanzar**
   errores propios con throw

3. **Gestionar**
   excepciones con try…catch…finally

Cinco tipos de errores

Tipo	Qué ocurre	Ejemplo
Sintaxis	Se detecta al escribir; el editor lo subraya	Una llave sin cerrar
Ejecución	Solo aparece al ejecutar	Llamar a una función no definida
Lógicos	Sintaxis correcta, resultado incorrecto	Un IVA mal calculado
Del sistema	Fuera de nuestro control	Caída de la red
De usuario	Entrada inesperada	Texto donde se pedía un número

Tipo 1 · Error de sintaxis

```
console.log("Inicio");
function saludar( {
    console.log("Hola");
}
```

QUÉ OCURRE

VS Code lo subraya en rojo. Al ejecutar: SyntaxError y ni siquiera se imprime “Inicio”.

IDEA CLAVE

El intérprete lee todo el archivo antes de ejecutarlo.

Tipo 2 · Error de ejecución

```
console.log("Inicio");
saludar();
console.log("Fin");
```

QUÉ OCURRE

El editor no marca nada. Se imprime “Inicio” y salta ReferenceError: saludar is not defined.

IDEA CLAVE

La sintaxis es correcta, pero lo que pedimos no existe. “Fin” no se imprime.

Tipo 3 · Error lógico

```
let a = 8, b = 10;
let media = a + b / 2;
console.log("Media:", media);
```

QUÉ OCURRE

Muestra 13 y no hay ningún aviso. La media correcta es 9.

IDEA CLAVE

Solo lo detecta quien sabe qué resultado debería salir. Se corrige con (a + b) / 2.

Tipo 4 · Error del sistema

```
const URL = "https://no-existe.invalid";
async function cargar() {
    try {
        await fetch(URL);
        console.log("OK");
    } catch (error) {
        console.log("Sin conexión");
    }
}
cargar();
```

QUÉ OCURRE

fetch falla porque la dirección no existe y se ejecuta el catch.

IDEA CLAVE

El código es correcto; el fallo es de la red. Solo controlamos cómo reacciona la aplicación.

Tipo 5 · Error de usuario

```
let edad = prompt("¿Edad?");
console.log(Number(edad) + 1);
```

QUÉ OCURRE

Si el usuario escribe abc, se muestra NaN y no hay ningún error.

IDEA CLAVE

Es un error lógico: no previmos esa entrada. Se soluciona con try…catch y throw.

¿Qué tipo de error es?

MINI-ACTIVIDAD · 5 MIN

1. Olvidé cerrar un paréntesis.
2. Llamo a calcular() y nunca la definí.
3. El carrito suma mal el IVA.
4. Se cae el wifi durante un fetch.
5. El usuario escribe “abc” en el campo de edad.

Crear y lanzar errores propios

```
let miError = new Error("Se esperaba un número");
throw miError;
```

Crear el objeto

No provoca ningún error.

Lanzarlo con throw

Provoca la excepción.

El bloque try…catch

```
try {
    // código que puede fallar
} catch (objetoError) {
    // si hay error
}
```

try contiene el código que puede provocar un error.

Si falla, el flujo salta a **catch**.

Las líneas restantes del **try** no se ejecutan.

Ejemplo 1: la variable no existe

```
try {
    console.log(e);
    console.log("aquí");
} catch (error) {
    let e = 1;
    console.log(e);
}
```

¿QUÉ SE MUESTRA?

```
1
```

"aquí" no aparece: el resto del try se salta.

Ejemplo 2: la variable ya existe

```
let e = 12;
try {
    console.log(e);
    console.log("aquí");
} catch (error) {
    let e = 1;
    console.log(e);
}
```

SALIDA

```
12
aquí
```

No hay error, así que el catch no se ejecuta.

El bloque finally

```
try {
    console.log(e);
    console.log("aquí");
} catch (error) {
    let e = 1;
    console.log(e);
} finally {
    console.log("Yo salgo siempre");
}
```

SALIDA

```
1
Yo salgo siempre
```

finally se ejecuta haya o no excepción.

Resumen

`try`	`catch`	`finally`	`throw`
prueba	captura	siempre se ejecuta	lanza

PREGUNTA DE SALIDA

Si el error ocurre en la línea 2 de un try de 5 líneas, ¿qué líneas se ejecutan?